Language Theory & Automata — Cheat Sheet

Dr-Ing Ines Baccouche · Pristini · 2nd year Applied AI · Chapters 1–3 complete (RegEx · Finite Automata, Glushkov & Minimisation · DFA / NFA) + extra methods + preview of Ch5–6 + 20 practice exercises

✍️ How she writes things in class — copy this format on the exam

- **Automaton** = 5-tuple `A = (Q, V_T, δ, q0, F)` — always write all 5 parts.
- **States**: `q0, q1…` or plain numbers `0,1,2…` (Glushkov). Initial = incoming arrow (in tables `→q0`). Final = **double circle** (in tables `*q2` or a **box** around it).
- **Transition table**: rows = states, columns = symbols, `δ` in the corner. Empty cell = `-` or `ϕ`. Several targets = a set `{q1, q2}`.
- **NFA→DFA**: one line per computation: `δ'([q1], 0) = [q1, q2]` `→ new state`, then **rename** the new set (`{q1,q2} = q3`) and redraw.
- **Glushkov**: number every letter with an index: `a1* b2 (a3|c4)* b5`. Then a **"successors" table** whose first row is `ϕ` (= start letters). Final letters are **boxed**.
- **Minimisation**: one line per step, groups in brackets: `Step0: (6,7,8) (1,2,3,4,5)` `Step1: (6) (7,8) (1,2,3,4,5)` … then "**fusion** (same behaviour)".
- **Identical rows** in a table → she **boxes them** → they are merged into one state.
- **Merged state** is named with all its members: `"1,2"`, `"7,8"`, `"0,1,3,6"`.

Chapter 1 — Language theory & Regular Expressions

1. Vocabulary

- **Symbol**: `a` · **Alphabet** Σ: finite set of symbols `{a, b}`
- **String / word**: finite sequence of symbols `aabb`
- **Language** L ⊆ Σ*: a set of strings `{ε, ab, aabb, aaabbb, …}` (finite or infinite)
- **ε** = empty string, `|ε| = 0` ; `|01101| = 5`
- **Σ^k** = strings of length k. $\Sigma^0 = \{\varepsilon\}$; Σ={0,1} ⇒ $\Sigma^2 = \{00,01,10,11\}$
- **Σ^*** = $\Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \dots$ (Kleene star, all strings)
- **Σ^+** = $\Sigma^1 \cup \Sigma^2 \cup \dots$ ⇒ $\Sigma^* = \Sigma^+ \cup \{\varepsilon\}$
- **Concatenation**: x=01101, y=110 → xy = 01101110, yx = 11001101. `εw = wε = w`

⚠ Classic traps

- **ø ≠ {ε}**: ø has *no* string, {ε} has *one* (the empty one).
- **Σ ≠ Σ^1**: Σ contains *symbols*, Σ^1 contains *strings* of length 1.
- xy ≠ yx in general (concatenation is not commutative).
- L* **always** contains ε; L^+ contains ε only if ε ∈ L.
- {0,11}* = 0s and 1s where the 1s come **in pairs**: 011, 11110, ε ✔ — 01011, 101 ✘

Exercise (slide 14) — A = {a,b}, u = aa, v = bab

- uv = **aabab** · $(uv)^2$ = **aababaabab** · u^3v = **aaaaaabab**
- Words of length 2: **{aa, ab, ba, bb}**
- E1 = {u.v / u,v ∈ A^+} = **words with |w| ≥ 2**; E2 = **A^+**; E3 = **A***

2. Operations on languages & RegEx symbols

Operation	Symbol	Meaning	Example
Union (alternation)	`|`	in L or in M (or both)	L={001,10,111}, M={ε,001} ⇒ L∪M = {ε,001,10,111}
Concatenation	`.` / juxtaposition	a word of L followed by a word of M	L.M = {001,10,111,001001,10001,111001}
Kleene star (closure)	`*`	0 or more repetitions (ε included)	`a*` → ε, a, aa, aaa…
Kleene plus (positive closure)	`+`	1 or more repetitions	`b+` → b, bb…; `\d+` → 1, 123…
Optional	`?`	0 or 1 time	`ab?` → a, ab
Exact / min count	`{n}` `{n,}`	exactly n / at least n times	`(a|b|c){3}` , `[a-z]{2,}`
Intersection · Difference · Complement	`n` · `\` · `¬`	common · in first not second · not in L	{a,b}∩{b} = {b}; {a,b}\{b} = {a}
Grouping	`( )`	order of evaluation	`(ab)*` → ε, ab, abab…

Priority: `*` (and `+` `?`) > `.` concatenation > `|` union. So `ab*|c` = `(a(b*)) | c`.

Regular expression (basis): ε and ø are RegEx (L(ε)={ε}, L(ø)=ø); every symbol a is a RegEx (L(a)={a}); if E, F are RegEx then E|F, EF, E* and (E) are RegEx. A language is **regular** ⇔ it is denoted by a RegEx.

3. RegEx notation (slide 33)

Token	Matches	Set
`\d`	any digit	`[0-9]`
`\s`	whitespace	`[ \t\n\r\f]`
`\w`	letter, digit, _	`[A-Za-z0-9_]`
`\D \S \W`	the inverse	`[^0-9]` ...
`.`	any one character	
`[A-Z]`	one capital letter	

UNIX example: `[A-Z][a-z]*[ ][A-Z][A-Z]` = capitalised word + space + 2 capitals (a city and a state: "Paris TX").

Uses: lexical analysers (compilers), search, text recognition, form validation.

4. Classic RegEx patterns over {a,b,c} (Exercise 2)

All strings	`(a|b|c)*` = `(a*b*c*)*`
Three-letter strings	`(a|b|c){3}`
Without 'b'	`(a|c)*`
At most one 'a'	`(b|c)* | (b|c)*a(b|c)*`
At least one 'a'	`(a|b|c)*a(a|b|c)*`
Each 'a' followed by 'b'	`(b|c|ab)*`
Even length	`((a|b|c)(a|b|c))*`
Prefix 'ab'	`ab(a|b|c)*`
Suffix 'ab'	`(a|b|c)*ab`
Exactly 3 'a'	`(b|c)*a(b|c)*a(b|c)*a(b|c)*`
At least 3 'a'	`(a|b|c)*a(a|b|c)*a(a|b|c)*a(a|b`

5. Solved exercises (slides 27–36)

R = a(a|b)ba = {aaba, abba}, S = (ab)*|(ba)*

1. In R not S: **aaba**
2. In S not R: **ab** (or ε, abab)
3. In both: **none!** R's words (length 4) are aaba, abba; S's words of length 4 are abab, baba ⇒ L(R)∩L(S) = ø
4. In neither: **b**, aa, bbb...

Give the elements:

- `ab*|c` = **{c, a, ab, abb, abbb, ...}**
- `(a+ b)` = **{ab, aab, aaab, ...}** (+ = one or more)
- `b*ab*ab*` = **words of {a,b} with exactly two a**: aa, aba, baab...
- `(a|b*)*(c|d)*` = **any word of a,b followed by any word of c,d**: ε, ab, ba, cd, abdc...

6. True or false? (slide 29)

- `(a*|b*) = (a|b)*` → **False**: *ab* ∈ (a|b)* but not in a*|b*
- `(a|b)*b = (a|b)*` → **False**: ε and *a* are not in (a|b)*b
- `(a*b)* = (a|b)*b` → **False**: ε ∈ (a*b)* but ε ∉ (a|b)*b. (True only if we add ε: (a*b)* = ε | (a|b)*b)

To prove "False" → give ONE word that is in one side and not the other (counter-example).

Exercises 3–5

- L1 = begins with a: `a(a|b)*` · L2 = even length: `((a|b)(a|b))*`
- 3-digit numbers: `\d{3}` · divisible by 5: `[0-9]*(0|5)` · start 4 end 7: `4[0-9]*7` (slide: `4.*7`)
- Email: `[a-zA-Z0-9]+@[a-zA-Z0-9]+\.[a-zA-Z]{2,}`

7. Simplifying a RegEx — the rules (identities)

not on her slides, but needed for "simplify" / "true or false"

Family	Rule
Union |	`r|s = s|r` · `r|r = r` · `r|ø = r` · `(r|s)|t = r|(s|t)`
Concatenation	`εr = rε = r` · `ør = rø = ø` · `r(s|t) = rs|rt` · `(s|t)r = sr|tr` · **rs ≠ sr**
Star basics	`ø* = ε` · `ε* = ε` · `(r*)* = r*` · `r*r* = r*`
Star + ε	`ε|rr* = r*` · `(ε|r)* = r*` · `(ε|r)r* = r*` · `ε|r* = r*`
Plus	`rr* = r*r = r+` · `r* = ε|r+` · `r+|ε = r*`
Sliding	`(rs)*r = r(sr)*` e.g. `(ab)*a = a(ba)*`
Big one	`(r|s)* = (r*s*)* = (r*|s*)* = (r*s)*r* = r*(sr*)*`
Absorption	if L(r) ⊆ L(s): `r|s = s`. e.g. `a | a* = a*`, `ab | (a|b)* = (a|b)*`
Star absorption	`r*(r|s)* = (r|s)*` · `(r|s)*r* = (r|s)*`

❌ **FALSE equalities (classic traps)**

- `(r|s)* ≠ r*|s*` (ab is a counter-example)
- `(r|s)* ≠ r*s*` (ba)
- `(rs)* ≠ r*s*` (aab is in a*b*, not in (ab)*)
- `r+ ≠ r*` (ε)

Worked simplifications

```
(a|ε)a*          = a*            (ε|r)r* = r*
ε | aa* | a      = a* | a = a*   absorption
a*(a|b)*         = (a|b)*        star absorption
(a|b)*(a|b)*     = (a|b)*        r*r* = r*
(a*b*)*a(a|b)*   = (a|b)*a(a|b)*
b*(ab*)*         = (a|b)*        r*(sr*)* = (r|s)*
(ab)*a | a(ba)*  = a(ba)*        sliding, then r|r = r
øa | b           = b             ør = ø, r|ø = r
```

How to check an equality fast: list all words of length 0, 1, 2 (and 3) on both sides. One word in one side only ⇒ **False**. If you want to say **True**, justify with the rules above.

Chapter 2 — Finite Automata principles · Glushkov · minimisation

1. Definition

A **finite automaton** (finite state machine) reads a string and answers **yes** if S ∈ L(G), **no** otherwise.

$$A = (Q, V_T, \delta, q0, F)$$

- V_T: finite alphabet · **Q**: finite set of states
- **q0**: initial state · **F**: set of final (acceptor) states
- **δ**: transition function $\delta : Q \times V_T \to Q$

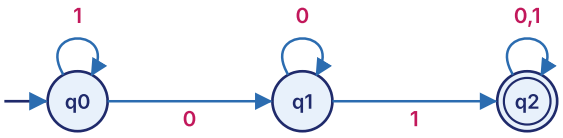
Accepted word: w = a1a2...an is accepted if there is a **path** that starts in the initial state, follows the labels a1, a2, ..., an and **ends in a final state**.

Used in: hardware design, lexical analysis (compilers), text search, robots/planning, program verification, text compression.

2. Example — strings over {0,1} containing "01"

She reasons with **scenarios → one state each**:

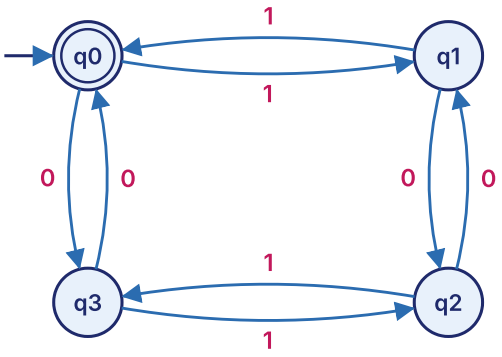
- q0: the first 0 has not appeared yet
- q1: a 0 appeared, waiting for a 1
- q2: "01" was met (final, stays forever)



3. Example — even number of 0 AND of 1

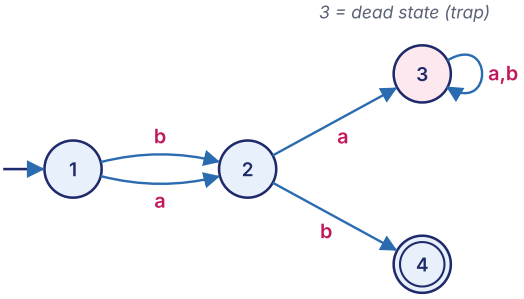
`M = ({q0,q1,q2,q3}, {0,1}, δ, q0, {q0})`

δ	0	1
→*q0	q3	q1
q1	q2	q0
q2	q1	q3
q3	q0	q2



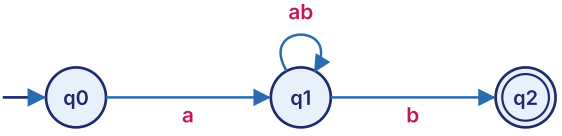
q0 = (even 0, even 1) · q1 = (even 0, odd 1) · q2 = (odd, odd) · q3 = (odd 0, even 1)

4. Reading a diagram (slide 49) & table (slide 47)



Σ={a,b}, Q={1,2,3,4}, F={4}. Language = **(a|b)b** = {ab, bb}. State 3 is a **trap**: once there, never accepted.

L = a(ab)*b, Q={q0,q1,q2}, V_T={a,b,ab}: δ(q0,a)=q1, δ(q1,b)=q2, δ(q1,ab)=q1



5. Is a word accepted? — how to write the trace

Extended function δ* (reads a whole word):

- `δ*(q, ε) = q` · `δ*(q, wa) = δ(δ*(q, w), a)`
- **L(A) = { w ∈ Σ* | δ*(q0, w) ∈ F }**

Trace of **1101** on the "contains 01" automaton:

```
(q0, 1101) ⊢ (q0, 101) ⊢ (q0, 01)
⊢ (q1, 1) ⊢ (q2, ε)   q2 ∈ F ⇒ accepted ✔
```

Short form also OK: `q0 –1→ q0 –1→ q0 –0→ q1 –1→ q2 ✔`

- **Rejected** if: the path ends in a non-final state, OR a transition is missing ("blocked").
- **NFA**: accepted if **at least one** path ends in a final state — try all choices.

Glushkov construction RegEx → automaton

Method — 3 steps

1. **Number** every symbol of the RegEx from 1 (each symbol appears once) → RE2.
   `a*b(a|c)*b → a1* b2 (a3|c4)* b5`
   States: q0 = 0, Q = {0,1,...,n} (n = number of letters).
2. **Build A'**:
   a. **Start letters** (can begin a word) → arcs `0 → i` labelled a_i
   b. **End letters** (can finish a word) → F = their numbers. **If ε ∈ L(RE) add 0 to F!**
   c. **Factors of length 2** a_ia_j (a_j can come right after a_i) → arc `i → j` labelled a_j
3. **Erase the indices** (a3 → a) and draw the automaton A.

Rule: an arc always goes INTO the state of the letter it reads: reading a_j ⇒ go to state j.

Glushkov for ANY expression — First / Last / Follow rules

For hard expressions, compute bottom-up: **N** = nullable (can give ε?), **First** = start letters, **Last** = end letters, **Follow** = factors of length 2.

E	N(E)	First(E)	Last(E)	new Follow
a_i	no	$\{a_i\}$	$\{a_i\}$	–
ε	yes	∅	∅	–
F | G	N(F) or N(G)	First(F) ∪ First(G)	Last(F) ∪ Last(G)	–
F G	N(F) and N(G)	First(F) ∪ First(G) *if N(F)*	Last(G) ∪ Last(F) *if N(G)*	Last(F) → First(G)
F*	yes	First(F)	Last(F)	Last(F) → First(F)
F+	N(F)	First(F)	Last(F)	Last(F) → First(F)
F?	yes	First(F)	Last(F)	–

Then: arcs **0 → First**, arcs **i → j** for each Follow pair, **F = Last** (+ **0** if N(E) = yes).

Check: number of states = number of letters + 1.

Other method: Thompson (RegEx → ε-NFA)

if asked

Build small blocks and glue them with ε-arrows (one start, one final per block):

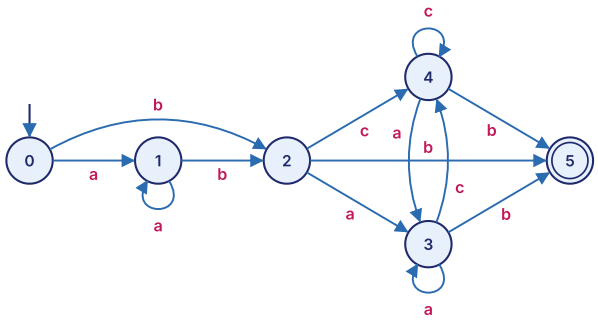
- **a**: i –a→ f
- **F|G**: new i –ε→ start(F) and start(G); final(F), final(G) –ε→ new f
- **FG**: final(F) –ε→ start(G)
- **F***: new i –ε→ start(F) and –ε→ new f; final(F) –ε→ start(F) and –ε→ f

Gives many ε-arrows → remove them with ε-closure (Ch3). Her course uses **Glushkov** (no ε, fewer states): use Glushkov unless the question says Thompson.

Example 1 — a*b(a|c)*b → a1* b2 (a3|c4)* b5

- Start letters = {a1, b2} · End letters = {b5} ⇒ F = {5}
- Factors: a1a1 a1b2 b2a3 b2c4 b2b5 a3a3 a3c4 c4a3 c4c4 a3b5 c4b5

	successors
ϕ	a1, b2
a1	a1, b2
b2	a3, c4, b5
a3	a3, c4, b5
c4	a3, c4, b5
b5	ϕ

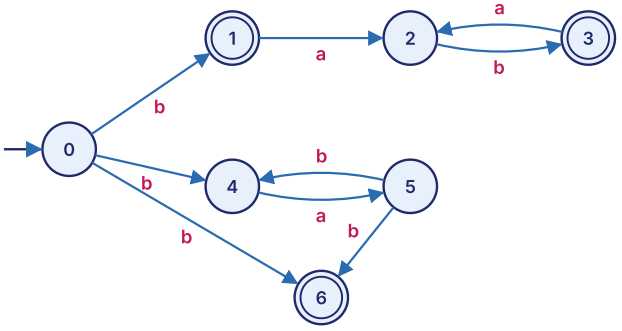


Example — b(ab)* | (ba)*b → b1(a2b3)* | (b4a5)*b6

	successors
ϕ	b1, b4, b6
b1	a2
a2	b3
b3	a2
b4	a5
a5	b4, b6
b6	ϕ

Boxed = end letters ⇒ **F = {1, 3, 6}**.
b1 is an end letter because (a2b3)* can be empty.

⚠ From 0, reading **b** goes to 1, 4 *and* 6 ⇒ Glushkov usually gives an **NFA** → determinise it (Ch3).



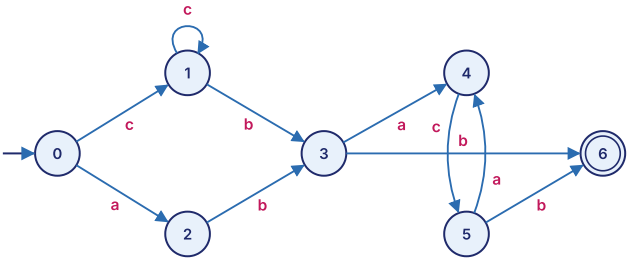
Example 2 (slide 57, left blank) — (c+|a) b (ac)* b — solved

`(c1+ | a2) b3 (a4 c5)* b6` (c+ = c c*, so c1 can follow c1)

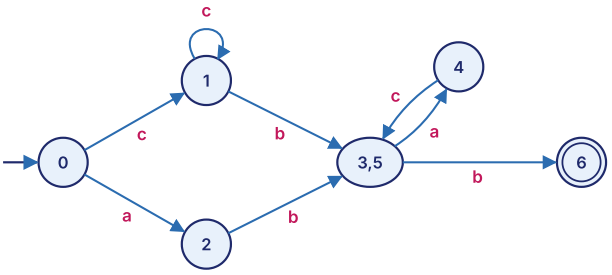
	successors
ϕ	c1, a2
c1	c1, b3
a2	b3
b3	a4, b6
a4	c5
c5	a4, b6
b6	ϕ

q0 = 0, Q = {0,…,6}, **F = {6}** (ε ∉ L).

Rows **b3** and **c5** have the same successors and both are non-final ⇒ states 3 and 5 can be **merged** (minimisation).



After minimisation (3 ≡ 5):



Minimisation of automata same language, fewest states

Method

Why: less memory/computation, easier to read; used in compilers, pattern matching, model checking.

1. **Remove unreachable states** (mark everything reachable from the start; delete the rest).
2. **Step0**: split into 2 groups → **(final states) (non-final states)**.
3. For every state of a group, look **which group** each letter sends it to. States of a group that go to **different groups** are **distinguishable** → split the group.
4. Repeat (Step1, Step2, ...) until nothing splits any more.
5. Each final group = **one state** ("fusion"). Initial = group containing the old initial; final = groups of old finals.

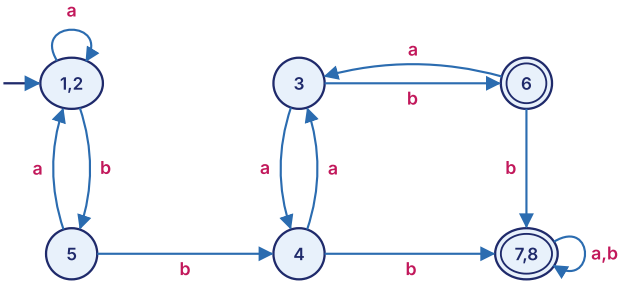
Two states are distinguishable if one word leads one of them to a final state and the other to a non-final one.

Example (slide 60) — F = {6, 7, 8}

	a	b
→1	1	5
2	2	5
3	4	6
4	3	8
5	2	4
*6	3	7
*7	8	7
*8	7	7

```
Step0 : (6,7,8)
(1,2,3,4,5)
Step1 : (6) (7,8)
(1,2,3,4,5)
Step2 : (6) (7,8) (3)
(4) (1,2,5)
Step3 : (6) (7,8) (3)
(4) (1,2) (5)
→ stable ⇒ 6 states
```

- Step1: 6 –a→ 3 (non-final) but 7,8 –a→ final ⇒ 6 alone.
- Step2: 3 –b→ 6 ∈ (6), 4 –b→ 8 ∈ (7,8), 1,2,5 stay inside ⇒ 3 and 4 alone.
- Step3: 5 –b→ 4 (now alone) but 1,2 –b→ 5 ⇒ 5 alone. 1 and 2 behave the same ⇒ merge.



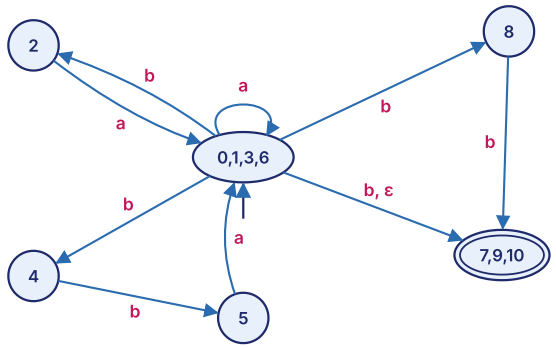
Class exercise (board photo, slide 61) — minimise (a | ba | bba)* (b | bb | ε)

① **Number:** `(a1 | b2a3 | b4b5a6)* (b7 | b8b9 | ε10)`

② **Successors table** (as on the board):

ϕ	a1 b2 b4 b7 b8 ε10
a1	a1 b2 b4 b7 b8 ε10
b2	a3
a3	a1 b2 b4 b7 b8 ε10
b4	b5
b5	a6
a6	a1 b2 b4 b7 b8 ε10
b7	ϕ
b8	b9
b9	ϕ
ε10	ϕ

Final letters ⇒ **F = {7, 9, 10}**



③ **Transition table** — boxed rows are identical:

	a	b	ε
→0	1	2,4,7,8	10
1	1	2,4,7,8	10
2	3	-	-
3	1	2,4,7,8	10
4	-	5	-
5	6	-	-
6	1	2,4,7,8	10
***7**	-	-	-
8	-	9	-
***9**	-	-	-
***10**	-	-	-

④ **Steps (her writing):**

```
1) {7,9,10} {0,1,2,3,4,5,6,8}
2) {7,9,10} {8} {2,4,5} {0} {1} {3} {6}
3) {7,9,10} {8} {2} {4} {5} {0} {1} {3} {6}
4) {7,9,10} {0,1,3,6} {8} {2} {4} {5}  ← fusion (same behaviour)
```

Why the fusions? 7, 9, 10 have no exit at all ⇒ same behaviour. 0, 1, 3, 6 have the **same row** (a→1, b→2,4,7,8, ε→10) ⇒ one state "0,1,3,6".

> **Note:** with strict step-by-step refinement 0,1,3,6 are never separated, and **2 and 5** also behave the same (each only reads *a* and goes back into {0,1,3,6}) — so they could be merged into "2,5" too. Write her version, and mention this if you have time.

Language check: "0,1,3,6" reads *a*, *ba* (via 2) or *bba* (via 4,5) and comes back; the end reads b, bb (via 8) or nothing (ε).

Chapter 3 — Deterministic & non-deterministic automata

1. Definitions

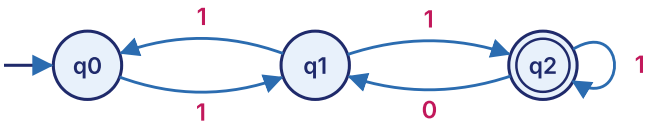
- **Completely defined**: for each state q and each symbol s ∈ Σ there is **at least one** transition labelled s leaving q.
- **Unambiguous**: for each state q and symbol s there is **at most one** transition labelled s leaving q.
- **DFA** (deterministic) = completely defined **AND** unambiguous ⇒ **exactly one** arrow per (state, symbol). **No ε-arrows.**
- **NFA** (non-deterministic) = presents **ambiguity**: several arrows with the same symbol from one state, or ε-arrows. "We cannot deduce the next state."

Formal NFA: `N = (Q, Σ, δ, q0, F)` with `δ : Q × (Σ ∪ {ε}) → P(Q)` (a **set** of states, possibly ø). For a DFA: `δ : Q × Σ → Q` (exactly one state).

How to complete a DFA: add a **dead state** (trap) that receives every missing transition and loops on all symbols.

Every NFA has an equivalent DFA (same language) → subset construction below.

2. Activity (slide 66) — solved



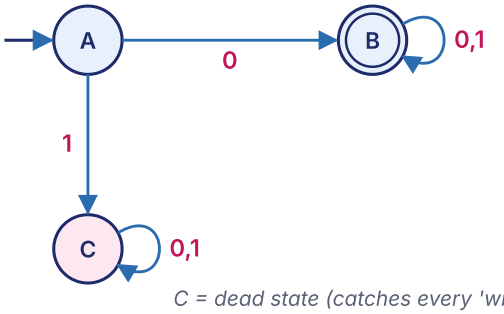
δ	0	1
→q0	-	q1
q1	-	{q0, q2}
*q2	q1	q2

- q1 –1→ q0 **or** q2 ⇒ ambiguity ⇒ **NFA**; q0 and q1 have no 0-arrow ⇒ not completely defined.
- **101**: q0 –1→ q1 –0→ ? no transition ⇒ **rejected (not modeled)**
- **1111**: q0→q1→q2→q2→q2 ⇒ **accepted**
- **1101**: q0→q1→q2 –0→ q1 –1→ q2 ⇒ **accepted**

3. Build a DFA from a sentence — her recipe

1. List the strings of L: L = {0, 00, 01, 000, ...}
2. Draw the **shortest accepted word** as a path.
3. Add the other arrows so that **every state has one arrow per symbol**.
4. Words that can never be accepted → **dead state**.

Starts with 0 over {0,1}:

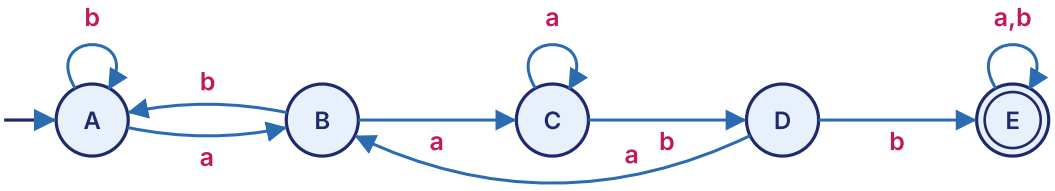


C = dead state (catches every 'wrong' word)

Length exactly 2: L = {00, 01, 10, 11}

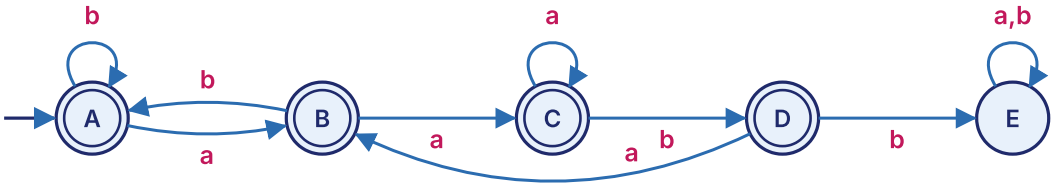


4. Contains "aabb" — and the complement trick



State = how much of "aabb" is matched (A: nothing, B: a, C: aa, D: aab, E: found). D –a→ B because "aaba" ends with "a".

Does NOT contain aabb → **flip the states**: 1. final → non-final, 2. non-final → final.



⚠ Flipping only works on a **complete DFA** (add the dead state first, never on an NFA).

NFA → DFA subset construction

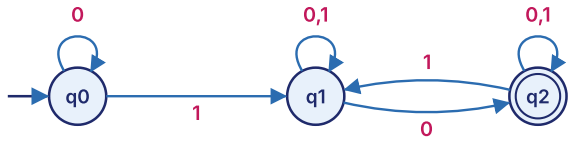
Method (M = (Q,Σ,δ,q0,F) → M' with L(M) = L(M'))

1. Q' = ϕ.
2. Add **[q0]** to Q' and compute its transitions.
3. For every state of Q' and every symbol, compute the **set** of reachable states: `δ'([q1,q2], 0) = δ(q1,0) ∪ δ(q2,0)`. If the set is **new** → add it to Q' ("new state").
4. Stop when no new state appears.
5. **Final states** of the DFA = every set that **contains** a final state of the NFA.

States never reached from [q0] (e.g. [q2] below) are **unreachable** → drop them. ϕ = dead state.

With ε-arrows: start from ε-closure(q0) (q0 + everything reachable by ε only), and after each move take the ε-closure again.

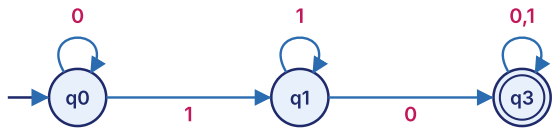
Example 1 (class photo, slides 75–77) — F = {q2}



NFA	0	1
→q0	q0	q1
q1	{q1,q2}	q1
*q2	q2	{q1,q2}

```
δ'([q0],0) = [q0]
δ'([q0],1) = [q1]
δ'([q1],0) = [q1,q2]
← new
δ'([q1],1) = [q1]
δ'([q1,q2],0) =
{q1,q2}∪{q2}
                =
[q1,q2]
δ'([q1,q2],1) = {q1}
∪{q1,q2}
                =
[q1,q2]
```

DFA	0	1
→q0	q0	q1
q1	{q1,q2} = q3	q1
*q3 = {q1,q2}	q3	q3

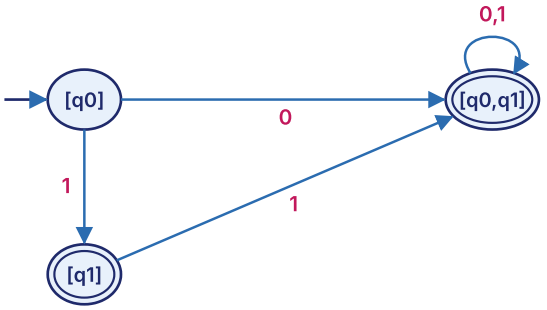


q3 = [q1,q2]

Example 2 (slide 78) — F = {q1}

	0	1
→q0	{q0,q1}	{q1}
*q1	ϕ	{q0,q1}

```
δ'([q0],0) = [q0,q1]
← new
δ'([q0],1) = [q1]
δ'([q1],0) = ϕ
δ'([q1],1) = [q0,q1]
δ'([q0,q1],0) =
[q0,q1]
δ'([q0,q1],1) =
[q0,q1]
```



[q1] and [q0,q1] are final (they contain q1). ϕ is the dead state (often not drawn).

ε-transitions: ε-closure and how to remove them

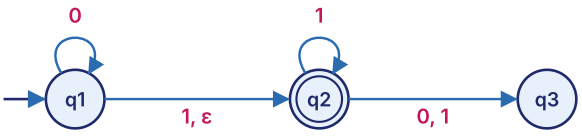
her slides use ε (slides 73, 79) — full method here

- **ε-arrow** = change state **without reading** any letter.
- **ε-closure(q)** = q itself + every state reachable from q using **only ε-arrows** (follow chains: q –ε→ p –ε→ r ⇒ r too).
- ε-closure of a set = union of the closures.

Subset construction with ε (= NFA→DFA + closures):

1. Start state of the DFA = **ε-closure(q0)**.
2. For a DFA state S and a letter x: **move** = all states reachable from S by one x-arrow, then take the **ε-closure** of that.
   `δ'(S, x) = ε-closure( ∪q∈S δ(q, x) )`
3. Repeat for every new set. Final = sets containing an NFA final state.

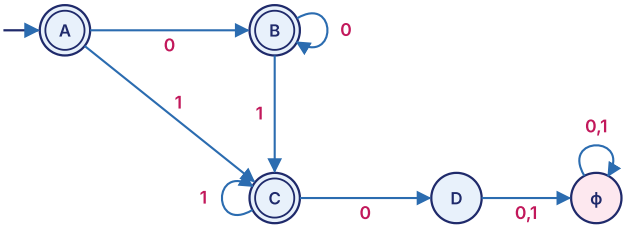
Example (slide 73, first NFA) — F = {q2}



ε-closure(q1) = {q1, q2} · ε-closure(q2) = {q2} · ε-closure(q3) = {q3}

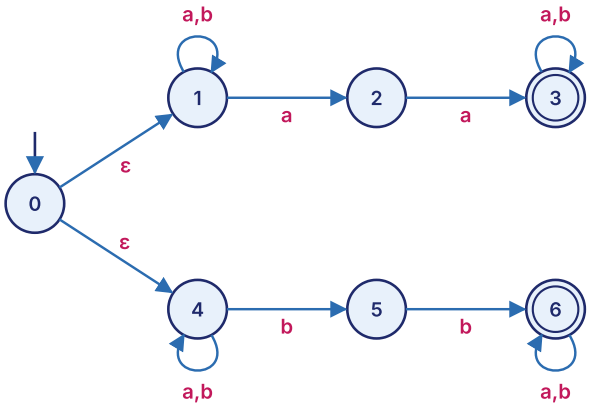
DFA state	0	1
→*A = {q1,q2}	{q1,q3}→cl = {q1,q2,q3} = B	{q2,q3} = C
*B = {q1,q2,q3}	B	C
*C = {q2,q3}	{q3} = D	C
D = {q3}	ϕ	ϕ
ϕ (dead)	ϕ	ϕ

```
A on 0: q1–0→q1, q2–0→q3 ⇒ {q1,q3}
        ε-closure ⇒ {q1,q2,q3}  (q1 –ε→ q2)
A on 1: q1–1→q2, q2–1→{q2,q3} ⇒ {q2,q3}
```



A is final because ε-closure(q1) contains q2: the empty word is accepted.

Application (slide 79) — ε-NFA → DFA — solved

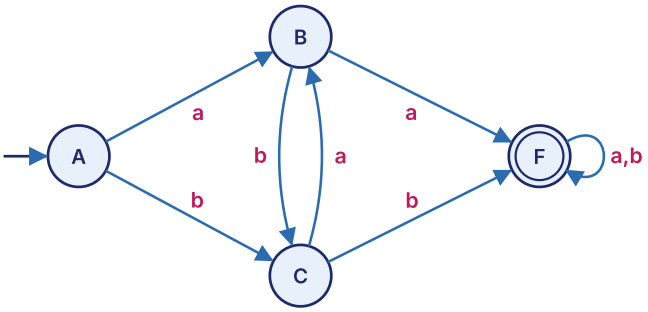


Language = words over {a,b} that contain **aa** or **bb**.

DFA state	a	b
→A = {0,1,4}	{1,2,4} = B	{1,4,5} = C
B = {1,2,4}	{1,2,3,4} = D	C
C = {1,4,5}	B	{1,4,5,6} = E
*D = {1,2,3,4}	D	{1,3,4,5} = F
*E = {1,4,5,6}	{1,2,4,6} = G	E
*F = {1,3,4,5}	D	{1,3,4,5,6} = H
*G = {1,2,4,6}	{1,2,3,4,6} = I	E
*H = {1,3,4,5,6}	I	H
*I = {1,2,3,4,6}	I	H

ε-closure(0) = {0,1,4} = **A** (start)

```
Minimise:
Step0: (D,E,F,G,H,I) (A,B,C)
Step1: (D,…,I) (A) (B) (C)
  B –a→ final, C –b→ final, A never
  ⇒ A, B, C all different
  D…I only go to D…I ⇒ fusion
  ⇒ one accepting state "F"
```



Example — moves on a 3×3 chessboard (slides 80–86)

States = squares 1…9; inputs **w** = move to an adjacent white square, **b** = adjacent black square. Start 1, final = opposite corner 9.

NFA	w	b		w	b
→1	2,4	5	6	2,8	3,5,9
2	4,6	1,3,5	7	4,8	5
3	2,6	5	8	4,6	5,7,9
4	2,8	1,5,7	*9	6,8	5
5	2,4,6,8	1,3,7,9			

DFA	w	b
→q0 = {1}	q1	q2
q1 = {2,4}	q3	q4
q2 = {5}	q3	q5
q3 = {2,4,6,8}	q3	q6
q4 = {1,3,5,7}	q3	q6
*q5 = {1,3,7,9}	q3	q2
*q6 = {1,3,5,7,9}	q3	q6

7 states q0…q6 like on her slide; finals = the sets containing 9.

Extra methods not (yet) on her slides — but classic exam questions

A. Automaton → RegEx (Arden's lemma)

Arden: if `X = rX | s` (and ε ∉ L(r)) then `X = r*s`.

1. One unknown X_q per state = "words that lead from q to a final state".
2. For each arrow q –a→ p write `aX_p` ; add `| ε` if q is final.
3. Solve from the final states backwards, using Arden for loops.
4. Answer = X_{q0}.

Example: "contains 01" (Ch2 §2)

```
X0 = 1X0 | 0X1
X1 = 0X1 | 1X2
X2 = 0X2 | 1X2 | ε = (0|1)X2 | ε
⇒ X2 = (0|1)*            Arden
⇒ X1 = 0*1(0|1)*         Arden
⇒ X0 = 1*0 0*1(0|1)*     Arden
RegEx = 1*00*1(0|1)*
```

Example: slide 49: X1 = aX2|bX2, X2 = aX3|bX4, X3 = (a|b)X3, X4 = ε ⇒ X3 = (a|b)*ø = ø (dead state!), X2 = b, **X1 = (a|b)b**.

B. Automaton → RegEx (state elimination)

1. Add a new start **i –ε→ q0** and a new final: every old final **–ε→ f**.
2. Remove the other states one by one. For each path **p –x→ q –y→ r** with a loop **z** on q, write on p→r: `(old label) | x z* y` .
3. Several arrows between the same 2 states → join with |.
4. At the end only i → f remains: its label is the RegEx.

Same "contains 01" example: remove q1 ⇒ q0 –00*1→ q2; remove q0 (loop 1) ⇒ i –1*00*1→ q2; remove q2 (loop 0|1) ⇒ **1*00*1(0|1)***. ✔ same as Arden.

C. Operations on automata

- **Complement** ¬L: complete DFA, then flip final / non-final.
- **Union / intersection** (product construction): states = pairs (p, q), one from each DFA; `δ((p,q), x) = (δ1(p,x), δ2(q,x))` ; start (p0, q0).
  Final for ∩: both final. Final for ∪: at least one final.
- **Concatenation, star**: easiest via RegEx (L1L2, L*) then Glushkov.

Example: L1 = contains a (P0 → P1*), L2 = ends with b (Q0 → Q1*)

	a	b	∩	∪
→(P0,Q0)	(P1,Q0)	(P0,Q1)		
(P0,Q1)	(P1,Q0)	(P0,Q1)		*
(P1,Q0)	(P1,Q0)	(P1,Q1)		*
(P1,Q1)	(P1,Q0)	(P1,Q1)	*	*

D. Regular or NOT regular?

Kleene's theorem: L is regular ⇔ described by a RegEx ⇔ recognised by a finite automaton (DFA = NFA = ε-NFA in power).

A finite automaton has **finite memory**: it cannot count without limit.

Regular ✔	NOT regular ✘
a*b* (a^nb^m)	{a^nb^n | n ≥ 0} = {ε, ab, aabb, ...}
even number of a	same number of 0 and 1
contains / starts / ends with "ab"	{0^i1^j | 0 ≤ i ≤ j}
number of a divisible by 3	palindromes, balanced parentheses
any finite language	{a^n | n prime}, {a^{n^2}}

Pumping lemma: if L is regular, there is p such that every w ∈ L with |w| ≥ p splits as w = xyz with |xy| ≤ p, |y| ≥ 1 and **xy^kz ∈ L for all k ≥ 0**.

Proof a^nb^n is not regular: take w = a^pb^p. |xy| ≤ p ⇒ y is only a's. Pump k = 2: xy^2z has more a's than b's ∉ L. Contradiction ⇒ not regular.

E. Chomsky hierarchy (where automata fit)

Type	Languages	Machine
3	Regular (RegEx)	Finite automaton
2	Context-free (a^nb^n, parentheses)	Pushdown automaton (stack)
1	Context-sensitive ($a^nb^nc^n$)	Linear bounded automaton
0	Recursively enumerable	Turing machine

Type 3 ⊂ Type 2 ⊂ Type 1 ⊂ Type 0: each machine is more powerful than the one above it.

F. Minimisation — the table (pairs) method

alternative

1. Draw the triangle of all pairs {p, q}.
2. Mark × every pair with one final and one non-final.
3. Repeat: mark {p,q} if for some letter x the pair {δ(p,x), δ(q,x)} is already marked.
4. Unmarked pairs = equivalent ⇒ merge.

Same result as her "Step" method; use hers in the exam.

Chapter 5 — Turing machine preview: her slides are not out yet, so her notation may differ

Definition

`M = (Q, Σ, Γ, δ, q0, B, F)`

- Q states · Σ input alphabet · **Γ tape alphabet** (Σ ⊂ Γ) · **B** = blank symbol
- q0 initial · F final (accepting) states
- `δ : Q × Γ → Q × Γ × {L, R}` : `δ(q, a) = (p, b, R)` = "in state q reading a: write b, move Right, go to p"

How it works: infinite tape, a read/write head. Unlike a FA it can **write** and move **both ways**. It accepts when it reaches a final state; it rejects when it is blocked (no transition) in a non-final state; it may also **loop forever**.

Configuration notation: `X a q1 b b` = tape "Xabb", head on the letter right after q1.

	FA	Turing machine
Memory	states only	infinite tape
Head	reads, moves right	reads + writes, L or R
Recognises	regular languages	everything computable

Classic example: L = {$a^n b^n$ | n ≥ 1}

Idea: mark the first a as **X**, go right to the first b, mark it **Y**, come back, repeat.

δ	a	b	X	Y	B
→q0	(q1,X,R)	-	-	(q3,Y,R)	-
q1	(q1,a,R)	(q2,Y,L)	-	(q1,Y,R)	-
q2	(q2,a,L)	-	(q0,X,R)	(q2,Y,L)	-
q3	-	-	-	(q3,Y,R)	(q4,B,R)
*q4	-	-	-	-	-

```
q0 aabb ⊢ X q1 abb ⊢ Xa q1 bb ⊢ X q2 aYb
⊢ q2 XaYb ⊢ X q0 aYb ⊢ XX q1 Yb ⊢ XXY q1 b
⊢ XX q2 YY ⊢ X q2 XYY ⊢ XX q0 YY ⊢ XXY q3 Y
⊢ XXYY q3 B ⊢ XXYYB q4   accepted ✔
```

q0 = find next a · q1 = go right to a b · q2 = go back left · q3 = check only Y's remain · q4 = accept.

Chapter 6 — Automata in AI preview

Where automata are used in AI

- **NLP**: tokenisers and lexers are DFAs built from RegEx; morphology (word forms) with finite-state transducers.
- **Data cleaning / extraction**: RegEx to find emails, dates, phone numbers before training a model.
- **Agents, robots, games**: behaviour as a finite state machine (states = Idle, Patrol, Attack; transitions = events).
- **Chatbots**: dialogue states (greeting → question → answer → goodbye).
- **Speech / sequences**: probabilistic automata, Hidden Markov Models (states + transition probabilities).
- **Verification**: model checking of critical / embedded systems, checking an AI policy never reaches a "bad" state.

Mealy / Moore machines (automata with output)

- **Moore**: output depends on the **state** only (e.g. traffic light: state Green ⇒ output "go").
- **Mealy**: output depends on **state + input**; written on the arrow as `input / output` (e.g. `coin / unlock`).
- Used for controllers, vending machines, protocol design.

Practice — new exercises, answers hidden tap "Answer" only after trying

Chapter 1 — RegEx

P1. Over {a,b}: RegEx for words with an odd number of a.

▼ Answer

`b*ab*(ab*ab*)*` — one a, then a's come in pairs. Words: a, ba, aaa, abab...

P2. Words over {a,b} that do not contain "aa".

▼ Answer

`(b|ab)*(ε|a)` — every a is followed by b, except maybe a last a.

P3. List the 5 shortest words of `(a|bb)*b`.

▼ Answer

b, ab, aab, bbb, aaab (lengths 1, 2, 3, 3, 4)

P4. True or false: `(ab)*a = a(ba)*` ? `(a|b)* = a*b*` ?

▼ Answer

1st **True** (sliding rule: both give a, aba, ababa...). 2nd **False**: ba ∈ (a|b)* but ba ∉ a*b*.

P5. Simplify: `(a|ε)*` · `ε | a*a` · `a*(a|b)* | b` · `(a*b)*a*`

▼ Answer

`a*` · `a*` · `(a|b)*` · `(a|b)*`

P6. RegEx for numbers (no leading 0 except "0") divisible by 10 — digits d = [0-9].

▼ Answer

`0 | [1-9][0-9]*0`

Chapter 2 — Glushkov & minimisation

P7. Glushkov automaton of `(a|b)*abb`.

▼ Answer

`(a1|b2)* a3 b4 b5`
ϕ: a1 b2 a3 · a1: a1 b2 a3 · b2: a1 b2 a3 · a3: b4 · b4: b5 · **b5**: ϕ ⇒ F = {5}
Arcs: 0,1,2 –a→ 1 and 3; 0,1,2 –b→ 2; 3 –b→ 4; 4 –b→ 5. It is an **NFA** (a goes to 1 and 3). Rows 0, 1, 2 are identical ⇒ merge as "0,1,2". See P13 for the DFA.

P8. Glushkov automaton of `(ab|b)*a`.

▼ Answer

`(a1b2 | b3)* a4` — ϕ: a1 b3 a4 · a1: b2 · b2: a1 b3 a4 · b3: a1 b3 a4 · **a4**: ϕ ⇒ F = {4}
Arcs: 0 –a→ 1, 0 –b→ 3, 0 –a→ 4; 1 –b→ 2; 2 and 3 have the same arcs as 0 (rows 0, 2, 3 identical ⇒ merge).

P9. Minimise (F = {E}). Remove unreachable states first.

	a	b
→A	B	C
B	B	D
C	B	C
D	B	E
*E	B	C
G	E	A

▼ Answer

G is unreachable (no arrow goes to G) ⇒ delete it.

```
Step0: (E) (A,B,C,D)
Step1: (E) (D) (A,B,C)      D –b→ E final
Step2: (E) (D) (B) (A,C)    B –b→ D
Step3: stable: A and C: a→B, b→C
```

Result: 4 states "A,C", B, D, E — it is the DFA of (a|b)*abb.

P10. With the DFA of P9: are abb, abab, babb accepted?

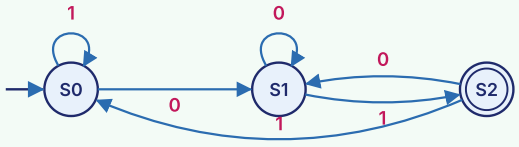
▼ Answer

abb: A→B→D→E ✔ · abab: A→B→D→B→D ✘ (D not final) · babb: A→C→B→D→E ✔

Chapter 3 — DFA / NFA

P11. DFA over {0,1}: words that *end with 01*.

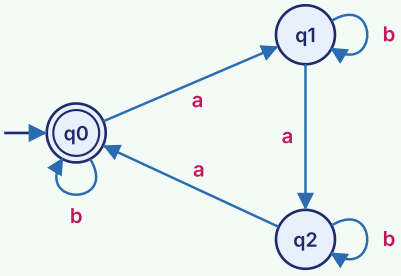
▼ Answer



S0 = nothing useful · S1 = last letter 0 · S2 = ends with 01.

P12. DFA over {a,b}: number of a *divisible by 3*.

▼ Answer

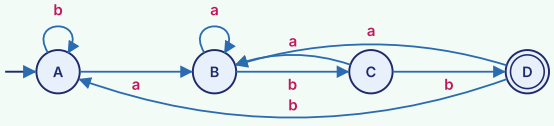


State = (number of a) mod 3; b changes nothing.

P13. Determinise the NFA of P7 (after merging 0,1,2 into S: S –a→ {S,3}, S –b→ S, 3 –b→ 4, 4 –b→ 5).

▼ Answer

```
δ'([S],a) = [S,3] ← new
δ'([S],b) = [S]
δ'([S,3],a) = [S,3]
δ'([S,3],b) = [S,4] ← new
δ'([S,4],a) = [S,3]
δ'([S,4],b) = [S,5] ← new
δ'([S,5],a) = [S,3]
δ'([S,5],b) = [S]
A=[S], B=[S,3], C=[S,4], *D=[S,5]
```

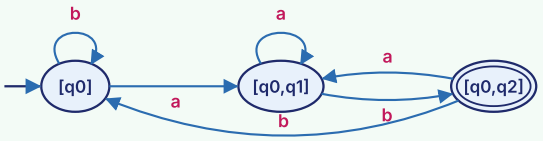


Same as P9's minimal DFA ✔

P14. NFA: q0 –a→ {q0,q1}, q0 –b→ q0, q1 –b→ q2, F = {q2}. Convert to a DFA. Which language?

▼ Answer

```
δ'([q0],a) = [q0,q1] ← new    δ'([q0],b) = [q0]
δ'([q0,q1],a) = [q0,q1]       δ'([q0,q1],b) =
[q0,q2] ← new
δ'([q0,q2],a) = [q0,q1]       δ'([q0,q2],b) = [q0]
```

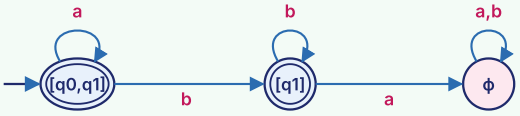


Language: words ending with **ab** = `(a|b)*ab`.

P15. ε-NFA: q0 –a→ q0, q0 –ε→ q1, q1 –b→ q1, F = {q1}. Convert to a DFA.

▼ Answer

ε-closure(q0) = {q0,q1} (final, contains q1). [q0,q1] –a→ cl{q0} = [q0,q1]; –b→ [q1]. [q1] –b→ [q1], –a→ ϕ.



Language: `a*b*`.

P16. DFA for "does NOT end with 01".

▼ Answer

Take P11 (complete DFA) and flip: S0, S1 final; S2 non-final.

Mixed / extra

P17. RegEx of the DFA of P12 (number of a divisible by 3) — use Arden.

▼ Answer

```
X0 = bX0 | aX1 | ε
X1 = bX1 | aX2      ⇒ X1 = b*aX2
X2 = bX2 | aX0      ⇒ X2 = b*aX0
⇒ X0 = bX0 | ab*ab*aX0 | ε = (b | ab*ab*a)X0 | ε
⇒ X0 = (b | ab*ab*a)*
```

P18. Is L = {$a^n b^{2n}$ | n ≥ 0} regular?

▼ Answer

No. Pumping lemma with w = $a^p b^{2p}$: y contains only a's, xy^2z has too many a's ∉ L.

P19. Run the Turing machine (Ch5) on *aab*.

▼ Answer

q0 aab ⊢ X q1 ab ⊢ Xa q1 b ⊢ X q2 aY ⊢ q2 XaY ⊢ X q0 aY ⊢ XX q1 Y ⊢ XXY q1 B → no transition, q1 not final ⇒ **rejected** ✔ (aab ∉ $a^n b^n$)

P20. Is this automaton deterministic? 1 –a→ 2, 1 –b→ 1, 2 –a→ 2, 2 –a→ 1, 2 –b→ 1.

▼ Answer

No: from 2, reading a goes to 2 **and** 1 ⇒ ambiguous ⇒ NFA (it is complete though).

Exam-day checklist

Which method for which question?

Question says…	Do…
"Give the RegEx of…"	Pattern table (Ch1 §4)
"Give the elements / words of…"	List the 4–5 shortest words + "…"
"Is this equality true?"	Counter-example word
"Build the automaton of this RegEx"	Glushkov (number → successors → draw)
"Minimise"	Remove unreachable → Step0 (F)(non-F) → split → fusion
"Is it deterministic?"	Check complete + unambiguous (+ no ε)
"Convert NFA to DFA"	Subset construction, final = contains NFA final
"Doesn't contain / complement"	Complete DFA + flip states
"Is word w accepted?"	Follow the path; must END in a final state
"Simplify the RegEx"	Identities table (Ch1 §7)
"Give the RegEx of this automaton"	Arden equations or state elimination
Automaton with ε-arrows	ε-closure + subset construction
"Is L regular?"	Build a FA/RegEx (yes) or pumping lemma (no)
"L1 ∩ L2", "L1 ∪ L2"	Product construction

⚠ Mistakes that cost points

- Forgetting to mark the **initial arrow** and the **double circles**.
- Glushkov: forgetting to add **0 to F when ε ∈ L**; forgetting that x* makes the next letter a possible start/end letter.
- Minimisation: starting with anything else than **(finals)(non-finals)**; not deleting unreachable states first.
- NFA→DFA: forgetting a "new state" in the list, or forgetting that a set is final if it contains **at least one** NFA final state.
- A DFA must have **exactly one** arrow per symbol in every state → add the dead state.
- Complementing an NFA by flipping states — **wrong**, determinise first.

Tools to check yourself: **regexper.com** (RegEx → diagram) · **automatonsimulator.com**

Coverage

✔ Everything in her 3 slide files (chapters 1–3) + all board exercises.
✔ Extra methods that exercises often need (simplification, ε-removal, automaton → RegEx, product, regular or not).
◂ Chapters 5–6 (Turing machine, automata in AI) are a **preview**: her slides are not out yet, so check her notation when they are.

Made from the course slides (chapter1_regEx, chapter2_automata, chapter3_DFA) and the board photos of 04/11. Print: Ctrl+P → A4.